What is PySpark?

PySpark is the Python API for Apache Spark.

It allows Python developers to process huge amounts of data across multiple systems very fast.

Simple meaning:

Python + Apache Spark = PySpark

PySpark is mainly used in:

- Data Engineering
- Big Data
- Data Analytics
- Machine Learning
- Streaming

What is Apache Spark?

Apache Spark is an open-source distributed computing engine used for processing huge amounts of data very fast.

It was developed to solve big data processing problems.

Spark processes data across multiple machines in parallel.

Simple Definition

Apache Spark is a big data processing framework that helps companies process massive datasets quickly using distributed computing.

Feature	PySpark	Apache Spark
Definition	Python API for Spark	Main distributed processing engine
Language	Python	Mainly Scala-based engine
Purpose	Allows Python users to use Spark	Core big data processing framework
Usage	Writing Spark applications using Python	Backend engine executing jobs
User Type	Python developers	Big data platform/framework

Feature	PySpark	Apache Spark
Coding Style	Python syntax	Scala/Java internals
Difficulty	Easier	More complex internally
Performance	Slightly slower than Scala Spark	Faster internally
Main Use	Data Engineering and Analytics	Distributed processing engine
Runs On	Top of Spark	Cluster systems

Why PySpark is Needed

Normal Python works well for:

- Small files
- Small datasets

But when data becomes huge:

- 100 GB
- 1 TB
- Petabytes

Normal systems become slow.

PySpark solves this problem using distributed computing.

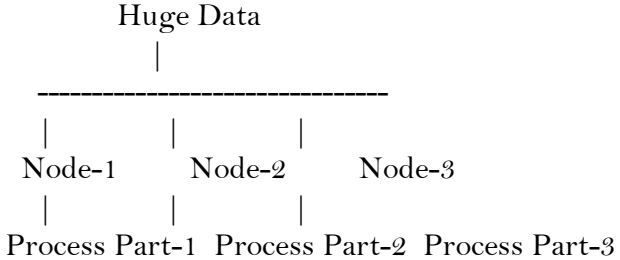
What is Distributed Computing?

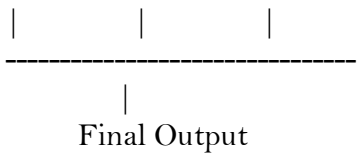
Distributed computing means:

- Splitting data across many systems
- Processing data in parallel

Instead of one computer doing all work, many machines work together.

Distributed Computing Diagram





Benefits of Distributed Computing

1. Speed

Multiple systems process data together.

2. Scalability

More systems can be added when data increases.

3. Fault Tolerance

If one system fails, others continue working.

What is Apache Spark?

Apache Spark is an open-source distributed computing engine.

It is designed for:

- Fast processing
- Big data analytics
- Real-time streaming
- Machine learning

Why Spark is Fast

Traditional systems:

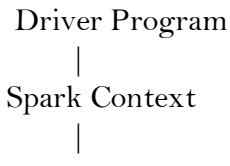
- Read/write from disk repeatedly

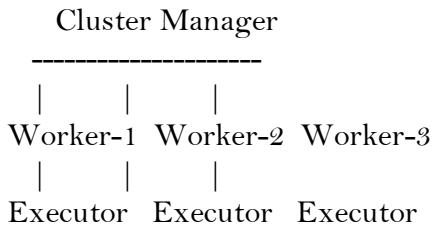
Spark:

- Stores intermediate data in memory (RAM)

This makes Spark much faster than Hadoop MapReduce.

Spark Architecture Diagram





Components Explanation

Driver Program

Main application controller.

Responsibilities:

- Starts application
- Sends jobs
- Monitors tasks

SparkContext

Connection between application and cluster.

Cluster Manager

Manages resources.

Examples:

- YARN
- Kubernetes
- Mesos

Worker Nodes

Machines that process data.

Executors

Run actual tasks on worker nodes.

What is PySpark?

PySpark is the Python interface for Spark.

It allows developers to use Spark using Python syntax.

Features of PySpark

Python Friendly

Easy to write using Python.

Big Data Processing

Handles TBs and PBs of data.

Rich Libraries

Supports:

- SQL
- Machine Learning
- Streaming

DataFrame Support

Works like SQL tables.

PySpark Modules

Module	Purpose
pyspark.sql	Structured data
pyspark.ml	Machine learning
pyspark.streaming	Real-time streaming
pyspark.graphx	Graph processing

What is RDD?

RDD = Resilient Distributed Dataset

RDD is:

- Distributed
- Fault tolerant
- Immutable

RDD Diagram

Original Data
|
Partitioned
|

| | | |
P1 P2 P3 P4

Each partition processes data independently.

What is DataFrame?

DataFrame is structured tabular data.

Similar to:

- Excel sheet
- SQL table
- Pandas DataFrame

Example:

Name	Age
Ram	25
Sam	30

Difference Between RDD and DataFrame

RDD	DataFrame
Low-level API	High-level API
Slower	Faster
No schema	Has schema
More coding	Easy coding

Lazy Evaluation

Spark transformations are not executed immediately.

Execution happens only after an action.

Lazy Evaluation Diagram

Transformation 1
|
Transformation 2
|
Transformation 3
|
Action
|
Execution Starts

Transformations in PySpark

Transformations modify data.

Examples:

- map()
- filter()
- flatMap()

Example:

rdd.map(lambda x: x*2)

No execution happens yet.

Actions in PySpark

Actions return final results.

Examples:

- collect()
- count()
- show()

Example:

rdd.collect()

Execution starts here.

PySpark Workflow

Read Data
|
Transform Data
|

Apply Business Logic
    |
Store Processed Data

Real Data Engineering Flow

S3 → PySpark → EMR → Redshift

Common services:

- Amazon Web Services S3
- EMR
- Redshift
- Glue

Basic PySpark Program

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Demo") \
    .getOrCreate()

data = [("Alice",25),("Bob",30)]

df = spark.createDataFrame(data, ["Name","Age"])

df.show()
```

Output

```
+-------+---+
| Name  |Age|
+-------+---+
| Alice |25 |
| Bob   |30 |
+-------+---+
```

Explanation

SparkSession

Entry point for Spark.

---

createDataFrame()

Creates structured DataFrame.

---

df.show()

Displays data.

Word Count Example

```
from pyspark import SparkContext

sc = SparkContext("local", "WordCount")

txt = "PySpark makes big data processing fast"

rdd = sc.parallelize([txt])

counts = rdd.flatMap(lambda x: x.split()) \
          .map(lambda word: (word,1)) \
          .reduceByKey(lambda a,b: a+b)

print(counts.collect())

sc.stop()
```

Word Count Flow Diagram

```
Sentence
   |
flatMap()
   |
Words
   |
map()
   |
(word,1)
   |
reduceByKey()
   |
Final Count
```

Explanation

parallelize()

Creates distributed dataset.

---

flatMap()

Splits sentence into words.

---

map()

Creates key-value pairs.

Example:

(PySpark,1)
(big,1)

---

reduceByKey()

Adds same word counts.

---

collect()

Returns results to driver.

Installing PySpark

Install Jupyter Notebook

pip install notebook

Install PySpark

pip install pyspark

Creating Spark Session

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyApp") \
    .getOrCreate()
```

Checking PySpark Version

Command Line

```
pyspark --version
```

Python Method

```
import pyspark

print(pyspark.__version__)
```

Environment Variables

JAVA_HOME

Stores JDK path.

SPARK_HOME

Stores Spark installation path.

Reading CSV Files

Single CSV File

```
df = spark.read.csv(
    "file.csv",
    header=True,
    inferSchema=True
)
```

Explanation

header=True

First row becomes column names.

inferSchema=True

Automatically detects data types.

Reading Multiple CSV Files

```
path = ["file1.csv","file2.csv"]

df = spark.read.csv(path, header=True)
```

Reading All CSV Files

```
df = spark.read.csv(
    "folder/*.csv",
    header=True
)
```

PySpark Processing Diagram

```
CSV Files
    |
Spark Read
    |
DataFrame
    |
Transformations
    |
Actions
    |
Final Output
```

Best Practices

Use DataFrames

Faster than RDDs.

Avoid collect() on Huge Data

May crash driver memory.

Use Partitioning

Improves parallel processing.

Cache Frequently Used Data

df.cache()

Real-Time Streaming

PySpark supports streaming using:

- Kafka
- TCP sockets
- Event systems

Machine Learning in PySpark

PySpark MLlib supports:

- Classification
- Regression
- Clustering

Platforms Running PySpark

Platform	Purpose
Hadoop	Big data cluster
Kubernetes	Container orchestration
Databricks	Managed Spark platform
AWS EMR	Cloud Spark cluster

Companies Using PySpark

- Walmart
- Netflix
- Amazon
- Facebook
- Trivago

Common Interview Questions

1. What is PySpark?
2. What is Spark?
3. Difference between RDD and DataFrame?
4. What is lazy evaluation?
5. What is partitioning?
6. What is caching?
7. What is shuffle?
8. Difference between Hadoop and Spark?
9. What is Spark Driver?
10. What is Executor?